Voice Assistant Query Normalizer and Text Cleaner

MT-AIML Mini Project — Problem 1

Student Name: _____

Enrollment Number: _____

Course: MT-AIML

Semester: _____

Academic Year: _____

Institution: _____

Submission Date: _____

Technical Documentation: Architecture, Methodology, and Controlled Evaluation

1. Problem Statement

Voice assistants commonly convert spoken requests into text before downstream command processing. Such raw speech-to-text transcripts may contain spoken fillers, acoustic or ASR annotation noise, contractions, casing variation, repeated words, punctuation, and function words that add little command value. Problem 1 addresses the deterministic NLP preprocessing of that transcript, including tokenization, stop-word handling, stemming/lemmatization comparison, and command-term preservation.

Input: a raw speech-to-text transcript supplied as text.

Output: normalized text intended for downstream voice-command processing. It is not a recognized audio signal and is not an execution/intent-classification result.

2. Objective

The project cleans raw voice-assistant transcripts; normalizes casing and configured contractions; removes selected fillers, ASR tags, and stop words; preserves protected command and negation terms; compares Porter and Lancaster stemming with WordNet and spaCy lemmatization; quantifies unique-vocabulary changes; analyzes stemming-error candidates; and supplies a runnable CLI, evaluation workflow, and demonstration UI.

3. Scope

The system starts with a transcript string or UTF-8 text file. The current Problem 1 implementation performs text preprocessing and controlled NLP comparison only. It does **not** record audio, decode audio files, perform speech recognition, or use Whisper, microphone input, or cloud speech services. A speech-to-text component is outside the current scope; audio integration is listed only as optional future work.

4. System Architecture

Raw speech-to-text transcript	Validation	Unicode + lowercase	Contraction expansion	Regex noise removal	Regex tokenization	Filler handling	Protected keyword detection	Stop-word filtering	Normalized query
>	>	>	>	>	>	>	>	>	>

Morphology is an analysis branch: it does not replace or mutate the normalized command.

Stemming	Lemmatization
Porter · Lancaster	WordNet · spaCy

Implementation modules: `main.py` (CLI), `app.py` (Streamlit demo), `src/preprocessing.py` (cleaner), `src/pipeline.py` (fast normalization/optional morphology), `src/stemmer_comparison.py`, `src/lemmatizer_comparison.py`, and `src/evaluation.py` (controlled dataset and reports).

5. Preprocessing Methodology

Each preprocessing stage is deterministic and timed separately in `clean_transcript()`. Its normalized output is the canonical query. Comparative stemming and lemmatization do not alter this output.

5.1 Unicode and casing normalization

Input is normalized with Python's Unicode NFKC normalization; curly apostrophe characters are mapped to the straight apostrophe used by the contraction rules. Text is lowercased. The compiled word-token expression uses Unicode-aware letter matching so words such as café, naïve, résumé, and München remain intact.

5.2 Contraction handling

A deterministic dictionary in `src/config.py` expands configured English contractions using a compiled case-insensitive regex. For example, `don't` becomes `do not`. Keeping the negator matters because removing “not” could reverse the intended command. Ambiguous possessives are intentionally not treated as contractions by a general rule.

5.3 Regex-based noise removal

The cleaner removes configured bracketed/angle-bracketed ASR annotations from `inaudible, noise, static, unintelligible, unk, unknown`, collapses repeated runs of selected punctuation, replaces unsupported non-linguistic symbols with spaces, and normalizes whitespace. These operations do not indiscriminately delete digits. The token pattern separately recognizes signed numeric values, decimals, clock times, and percentages. Arbitrary unbracketed words are not assumed to be ASR noise.

5.4 Filler handling

Default single-token filler vocabulary is `ah, er, erm, hmm, uh, um`. Prolonged forms matching the explicit filler pattern (for example, “uhhh” and “ummm”) are counted as removed fillers. The default phrase list is empty; “you know” is not removed unconditionally because it can be meaningful. Filler words and phrases can be configured without rewriting the cleaning pipeline.

5.5 Tokenization

The production tokenizer is a precompiled Unicode-aware Python regular expression. It emits lowercased word tokens and preserves numeric tokens such as 22, 22.5, 7:30, 50%, and 204 as applicable. An NLTK `word_tokenize(..., preserve_line=True)` alternative is timed in the evaluation. These tokenizers can differ on percent signs: the production regex retains 50% as one semantic numeric token, while NLTK may split it into 50 and %.

5.6 Stop words and command vocabulary

The general stop set is `a, an, the`; the domain politeness set is `kindly, please`. These are intentionally small and are not the NLTK English stop list. Fillers are a separate category and are counted separately. Stop-word removal runs only after protected command words have been identified.

5.7 Protected command keywords

Configured protected vocabulary is `call, cancel, cannot, close, decrease, delete, disable, enable, increase, never, next, no, not, off, on, open, pause, play, previous, search, send, set, show, start, stop, turn`. Words such as on/off, open/close, play/pause, next/previous, increase/decrease, and enable/disable can encode the requested action or state. The cleaner exempts these tokens from stop-word removal and adjacent-duplicate collapse. Negation words are protected as well. This is a safety heuristic, not a proof of semantic correctness for every possible utterance.

5.8 Repeated words and empty output

Adjacent repeated ordinary words are collapsed, while repeated protected commands, negations, and recognized numeric/time values are retained. Punctuation-only input can yield no linguistic tokens; the pipeline reports an empty normalized result and skips optional morphology rather than presenting an empty command as usable output.

6. Stemming

Stemming applies rule-based suffix transformations to reduce related surface forms to a common stem. A stem can be truncated or not be a dictionary lemma. Here both stemmer branches receive the same normalized token sequence from the evaluation corpus.

6.1 Porter Stemmer

The implementation uses NLTK's `PorterStemmer`. On this controlled vocabulary it changed 16 distinct word types and reduced the unique vocabulary by 1 term(s). Stemming is deterministic and lightweight, but can produce non-lemmas and conflations; this single benchmark does not establish general accuracy or superiority.

6.2 Lancaster Stemmer

The implementation uses NLTK's `LancasterStemmer`, an iterative rule-based stemmer. On this run it changed 27 distinct types and reduced the unique vocabulary by 2 term(s). Its transformations can be more aggressive on some words; a larger reduction does not mean better semantic preservation. Neither stemmer is universally preferable based on these measurements.

7. Lemmatization

Lemmatization maps an inflected form toward a lexical lemma, using lexical and/or linguistic information rather than only trimming suffixes. It is still a model/tool output and is not a semantic command parser.

7.1 WordNet Lemmatization

The WordNet branch uses NLTK's `WordNetLemmatizer`. Penn Treebank POS tags are mapped to WordNet POS when the tagger resource is available; otherwise the implementation uses its documented lightweight suffix heuristic and noun fallback. In the current controlled run, its unique vocabulary reduced by 1 term(s). The example table shows its measured output for the temperature command; this is morphological output, not command execution.

7.2 spaCy Lemmatization

The spaCy branch loads `en_core_web_sm` once per process, disabling parser and named-entity components, then processes the normalized tokens. The evaluation run retained 71 unique terms for this particular benchmark. This reflects this dataset and model output only; it does not establish that spaCy always retains vocabulary or is semantically more accurate.

8. Vocabulary Metrics

Original vocabulary is the number of unique normalized terms before the morphology transformation. **Resulting vocabulary** is the number of unique terms after the transformation. Duplicate token occurrences do not increase either size.

Vocabulary reduction = Original Vocabulary − Resulting Vocabulary
Vocabulary reduction (%) = ((Original Vocabulary − Resulting Vocabulary) / Original Vocabulary) × 100
Vocabulary retention (%) = (Resulting Vocabulary / Original Vocabulary) × 100

Reduction and retention are complementary for non-empty input vocabularies. They quantify unique-type compression, not semantic accuracy, command correctness, or general NLP quality. For an empty original vocabulary, the implementation reports 0.0% for both rates to avoid division by zero and does not imply a meaningful ratio.

9. Comparative Analysis — Latest Generated Evaluation

The following values are read directly from `reports/stemming_lemmatization_comparison.csv` after the latest evaluation run used to generate this document. Execution times are observed timings from this environment and are not fixed performance guarantees.

Method	Original Vocabulary	Resulting Vocabulary	Reduction	Reduction %	Retention %	Execution Time (s)
Porter	71	70	1	1.408451%	98.591549%	0.000890800
Lancaster	71	69	2	2.816901%	97.183099%	0.000894800
Wordnet	71	70	1	1.408451%	98.591549%	0.004197000
Spacy	71	71	0	0.000000%	100.000000%	0.009552100

10. Stemming Error Analysis

10.1 Over-stemming

Over-stemming occurs when distinct words that should remain distinct are mapped to the same stem. The evaluation detects candidates by grouping distinct normalized benchmark word types by their actual stem. A collision is marked as a manually verified unrelated pair only if it matches the manually curated contrast university/universal and the method actually conflates that pair.

Method	Word A	Word B	Shared Stem	Evidence	Manual Relation	Verified Error?
Porter	universal	university	univers	manual	unrelated	True
Lancaster	time	timer	tim	automatic_collision_candidate	unreviewed	False
Lancaster	universal	university	univers	manual	unrelated	True

The “university” / “universal” pair is a curated unrelated contrast; the report records a verified conflation only when observed for that stemmer. Automatic collisions such as “time” / “timer” remain unreviewed candidates, not confirmed semantic errors. Candidate counts depend on benchmark vocabulary.

10.2 Under-stemming

Under-stemming candidates arise when a manually curated related pair remains different after stemming. The current candidate set uses the irregular pairs goose/geese and mouse/mice. Their actual stems and method are read from `reports/error_analysis.csv`. Different stems are reported as candidates; they are not automatically labeled errors because the intended grouping depends on the application.

Method	Word A	Word B	Distinct Stems	Curated Relation	Status
Porter	goose	geese	goos|gees	related_irregular_forms	Candidate
Porter	mouse	mice	mous|mice	related_irregular_forms	Candidate
Lancaster	goose	geese	goos|gees	related_irregular_forms	Candidate
Lancaster	mouse	mice	mous|mic	related_irregular_forms	Candidate

11. Example Transformations

Each row below is generated by executing the current pipeline with optional morphology enabled. Output is shown as token sequences for the four comparison methods; unavailable resources would be labelled rather than synthesized.

Raw Transcript	Normalized Transcript	Porter	Lancaster	WordNet	spaCy
Uh PLEASE turn OFF the living room light!!!	turn off living room light	turn off live room light	turn off liv room light	turn off live room light	turn off living room light
don't turn off the fan	do not turn off fan	do not turn off fan	do not turn off fan	do not turn off fan	do not turn off fan
set temperature to 22.5 degrees	set temperature to 22.5 degrees	set temperatur to 22.5 degre	set temp to 22.5 degr	set temperature to 22.5 degree	set temperature to 22.5 degree
set an alarm for 7:30	set alarm for 7:30	set alarm for 7:30	set alarm for 7:30	set alarm for 7:30	set alarm for 7:30
open the café door in München	open café door in münchen	open café door in münchen	op café door in münchen	open café door in münchen	open café door in münchen
do you know the time?	do you know time	do you know time	do you know tim	do you know time	do you know time

12. Semantic Preservation

Regression tests and pipeline runs cover turning a light on/off, preserving “do not” after contraction expansion, retaining command verbs, numbers including 22.5, and the clock token 7:30. The default cleaner removes articles and configured politeness terms but does not remove on/off, protected command verbs, or negation. Unicode examples preserve café and München in the normalized output. These are specific tested cases, not a guarantee of universal semantic preservation.

Command keywords encode action and polarity; numbers, times, identifiers, and names can identify the object or parameter. Stemming and lemmatization are shown as analysis outputs only and must not be substituted for the canonical normalized command without application-specific review.

13. Evaluation Dataset

The controlled, manually assembled dataset is `data/evaluation/benchmark.csv` and contains **30 transcripts**. It covers command forms, fillers, casing, punctuation, contraction/negation, repeated terms, integer/decimal/time/percentage values, ASR tags, Unicode vocabulary, and curated morphology pairs. It is a reproducible coverage fixture, not a representative sample of Alexa/Siri or all real-world voice interactions.

14. Performance and Cleaning Statistics

Measured item	Latest report value	Definition
Input transcripts	30	Number of controlled dataset rows.
Tokens before cleaning	152	After Unicode/contraction/noise processing; before filler/repeat/stop filtering.
Tokens after cleaning	120	Final normalized tokens across all rows.
Token reduction (%)	21.052632	100 × (before – after) / before.
Fillers removed	6	Configured filler occurrences.
Stop words removed	24	General/domain stop-word occurrences.
ASR noise tags removed	1	Recognized bracketed artifact count.
Protected terms retained	37	Protected keyword occurrences in cleaned output.
Preprocessing time (s)	0.000653800	Sum of measured cleaner stage timings; morphology/reporting excluded.
Mean preprocessing time (s/transcript)	0.000021793	Total stage time / transcript count.
Regex tokenization time (s)	0.000094600	Aggregate production-tokenizer timing.
NLTK tokenization time (s)	0.000867300	Aggregate comparison timing; first-call setup may be included.
Tokenizer outputs equivalent	False	Whether all token sequences matched.

Morphology timings cover the transformation passes after WordNet resources and the spaCy model are loaded. Overall process startup and model loading are separate costs. All timings depend on CPU, Python, dependency versions, initialization state, and system load; they are measurements of this run, not guarantees.

15. Limitations

The controlled benchmark is small and curated. Regex, contraction, filler, stop-word, and duplicate rules are deterministic heuristics and may not fit every domain or dialect. Repeated ordinary words can sometimes encode emphasis. A protected vocabulary reduces specific risks but cannot guarantee intent preservation in arbitrary text. Stems can be malformed or conflate meanings; WordNet POS tagging depends on NLTK resources; spaCy lemmatization depends on its separately installed English model. No gold-standard semantic annotations are provided, so the report does not claim semantic accuracy. Upstream ASR errors and audio handling are out of scope.

16. Conclusion

The implementation provides a runnable deterministic transcript cleaner, keeps command, negation, and tested numeric forms in the normalized output, and generates reproducible controlled comparisons for Porter, Lancaster, WordNet, and spaCy. Evaluation measures vocabulary size changes and timed transformation passes; it does not establish real-world assistant accuracy or perfect semantic safety.

17. Future Scope

Potential follow-on work includes broader manually annotated text benchmarks, domain-specific review of filler and stop-word policies, multilingual evaluation, and explicit downstream command-semantic tests. Optional audio integration would be a separate upstream extension, not part of this submission:

Audio (future extension) > Speech-to-Text (future component) > Problem 1 text preprocessing

18. Reproducibility and Execution

Install Python dependencies with `python -m pip install -r requirements.txt`; install NLTK WordNet/tagger data with `python -m nltk.downloader wordnet omw-1.4 averaged_perceptron_tagger_eng`; and install spaCy's model with `python -m spacy download en_core_web_sm`. Run a single transcript with `python main.py --text "..."`, an input file with `python main.py --file data/raw_transcripts/sample.txt`, the controlled benchmark with `python main.py --evaluate`, tests with `python -m unittest discover -s tests -v`, and the UI with `python -m streamlit run app.py`.